

Architectural Blueprint and Modernization Report for the EduCore ERP Platform

Executive Summary and Strategic System Philosophy

The architectural evolution of modern educational management systems necessitates a fundamental transition from static, table-driven data repositories to dynamic, hierarchical knowledge graphs. The EduCore platform is designed as an AI-native, multi-tenant Software-as-a-Service (SaaS) ecosystem explicitly engineered for the complex compliance, academic, and financial environments of the Indian education sector. The foundational philosophy mandates that every entity, business process, and state transition serves as a node within a larger intelligence graph. This blueprint acts as the engineering source of truth, dictating the autonomous generation of application programming interfaces (APIs), user interfaces (UI), analytical data models, and automated compliance workflows.

The system is governed by a strict zero-trust philosophy, integrating the National Institute of Standards and Technology (NIST) Cybersecurity Framework (CSF) 2.0 with the stringent data protection requirements of the Digital Personal Data Protection (DPDP) Rules 2025.

Architecture decisions prioritize immutability, temporal querying via event sourcing, and rigid multi-tenant isolation through dual-layer schema and row-level security (RLS) enforcement. Furthermore, as an AI-orchestrated platform, the system operates under the constraints of the OWASP GenAI Security Project guidelines, specifically mitigating risks associated with excessive agency (LLM06) in autonomous systems.

The proposed design separates the presentation layer (web and mobile UI) from a centralized API gateway and loosely coupled backend microservices running in containers. The data layer utilizes a hybrid, polyglot storage model: a relational database for transactional consistency, a native property graph for relational depth and causal analysis, and a machine learning feature store to manage inputs for predictive models. The orchestration of these containerized services is managed via Kubernetes combined with a lightweight service mesh to handle cross-cutting concerns such as mutual Transport Layer Security (mTLS), distributed tracing, and traffic policy enforcement. Automated continuous integration and continuous deployment (CI/CD) pipelines coupled with deep observability metrics guarantee fast, resilient deployments.

Multi-Layered Architecture and Bounded Contexts

The domain architecture of the platform structures the enterprise resource planning (ERP) system not as a collection of disjointed tables, but as a hierarchical knowledge graph. This ontology enables retrieval-augmented generation (RAG), impact analysis, and cross-module reasoning by autonomous artificial intelligence (AI) agents. The institutional universe originates at the root tenant node and cascades downward through specialized operational domains. The primary nodes consist of the Institution at the apex, branching into Academic Year, Campus, and Department. These further subdivide into Class, Section, and Subject, while human actors are mapped as Student, Parent, Teacher, and Staff nodes. Operational facets are categorized into

Transport, Hostel, Library, Examination, Finance, HR, Assets, and Compliance, all feeding into Analytics and AI Knowledge nodes.

Presentation Layer and User Experience Blueprints

The presentation layer is composed of responsive, stateless Single Page Applications (Next.js 15 and React 19) utilizing the App Router to provide Server-Side Rendering (SSR) for optimal performance and search visibility where appropriate. The visual design system is standardized through TailwindCSS v4 and Radix UI primitives to ensure accessibility and rapid iteration during the AI-assisted code generation phases. Frontends communicate exclusively with the API gateway over HTTPS, relying on edge caching and access control at the ingress boundary. The user experience blueprint documents the psychological and operational reality of every role, focusing on goals, frustrations, frequency of interactions, and specific Key Performance Indicators (KPIs). For example, the daily attendance interface for teaching staff represents the highest frequency interaction point; consequently, the user interface is optimized to require a maximum of three interactions to submit class attendance and must support robust offline capabilities to accommodate network instability in rural regions.

Persona Component	Parent Persona Specification
Goals	Ensure ward's safety, monitor academic progress, and manage financial obligations seamlessly.
Frustrations	Delayed bus arrivals, lack of official communication channels, and physical bank visits for fees.
Interaction Frequency	Daily (Attendance, GPS), Monthly (Fees), Terminally (Report Cards).
Devices	Primary: Mobile (Progressive Web App / React Native).
Screen Flow	Dashboard \rightarrow Attendance \rightarrow Homework \rightarrow Fee Due \rightarrow Payment \rightarrow Receipt \rightarrow Transport GPS \rightarrow Teacher Chat.
Decision Points	Selecting EMI vs. UPI for fee payments; approving elective subjects.
Drop-off Points	Complex scholarship application forms; payment gateway timeouts.
AI Assistance	Predictive nudges for upcoming assignments; conversational interfaces for policy queries.
Automation	Auto-generation of fee receipts upon successful Razorpay webhook validation.
Required Approvals	Consent for external field trips; acknowledgment of disciplinary notices.
Key Performance Indicators	Time to complete fee payment (< 2 minutes); Time to locate school bus (< 10 seconds).

Centralized API Gateway and Ingress Controller

The API layer is managed by the Kong API Gateway, which serves as the single entry point for

all external traffic. The gateway coordinates global rate limiting, enforces Web Application Firewall (WAF) rules, and offloads authentication. It isolates external clients from the internal microservices topology, translates external REST/JSON requests to internal gRPC protocols where appropriate, and implements the Backend-for-Frontend (BFF) pattern to aggregate downstream responses, thereby reducing client-side roundtrips.

Microservice Layer and Bounded Context Enforcement

Independent microservices implement core educational domains, with each service maintaining its own codebase to scale separately. Domain integration risk analysis reveals that deeply coupled systems are susceptible to cascading failures; for instance, an error in the Academic domain (such as an incorrect promotion status) can corrupt the Financial domain (incorrect fee structure generation). To mitigate this risk, the domain model strictly enforces bounded contexts. Microservices communicate exclusively via asynchronous event streams using EventStoreDB, ensuring that state changes in one domain do not synchronously block or corrupt adjacent domains.

Access to these services is strictly tied to a complex matrix of user personas. The identity layer leverages JSON Web Tokens (JWT) signed with RS256 asymmetric cryptography, and the permission resolution algorithm extracts the tenant_id and the assigned role from the verified token payload before any API route is invoked.

Tier	Persona Designation	Primary Access Scope
Tier 0	Platform Super Admin	Full system access across all tenants, global billing, and global feature flag configuration.
Tier 1	Institution Admin / Owner	Full access within a single institution across all departments and campuses.
Tier 2	Principal / Vice Principal	Academic leadership, cross-departmental academic data, and staff performance oversight.
Tier 2	Head of Department (HOD)	Department-scoped academic oversight, lesson plan tracking, and subject performance.
Tier 3	Teacher (Class/Subject)	Class-specific academic data, attendance marking, homework distribution, and grading.
Tier 3	Accountant / Finance Officer	Full access to the financial module, fee collections, payroll, and GST reporting.
Tier 3	HR Manager	Staff lifecycle management, payroll initiation, and DPDP employee consent tracking.
Tier 3	Transport / Hostel / Library	Operational management limited strictly to their respective non-academic

Tier	Persona Designation	Primary Access Scope
		domains.
Tier 4	Student	Read-only access to personal academic performance, attendance, and assignment submission.
Tier 5	Parent / Guardian	Read-only access scoped strictly to their ward's data, fee payments, and GPS tracking.
Tier 6	External Auditor / Driver	Temporary or highly restricted functional access (e.g., read-only compliance views or GPS routing).

Hybrid Polyglot Persistence Layer and Schema Design

To prevent operational database exhaustion during analytical queries involving multiple joins across millions of rows, the system segregates the entity universe into an Online Transaction Processing (OLTP) core powered by PostgreSQL 17, and an Online Analytical Processing (OLAP) projection engine synchronized to ClickHouse. Operational high-velocity tables such as `attendance_daily`, `library_transactions`, and `hostel_allotments` write directly to a Valkey caching layer (a Linux Foundation Redis fork). High-frequency telemetry, particularly `gps_telemetry` from transport vehicles, is ingested into memory, batch-processed, and flushed to time-partitioned PostgreSQL tables to prevent I/O bottlenecks.

PostgreSQL Relational Schema Blueprints

The core transaction ledger leverages PostgreSQL 17 JSONB capabilities, maintaining absolute referential integrity while enforcing write-level immutability rules.

-- Core Tenants Schema

```
CREATE TABLE tenants (
    tenant_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    domain_name VARCHAR(255) NOT NULL UNIQUE,
    kms_dek_arn VARCHAR(512) NOT NULL, -- Reference to cryptographic
DEK in AWS KMS
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

-- Immutable Financial Ledger Table

```
CREATE TABLE fee_transactions (
    transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(tenant_id) ON DELETE
CASCADE,
    student_id UUID NOT NULL,
    amount_paise BIGINT NOT NULL, -- Currency handled strictly as
```

```

integers in paise
    currency VARCHAR(3) DEFAULT 'INR',
    payment_status VARCHAR(50) NOT NULL CHECK (payment_status IN
('PENDING', 'INITIATED', 'PROCESSING', 'CAPTURED', 'FAILED',
'REFUNDED')),
    audit_metadata JSONB, -- Schema-less dimension holding security
and context metadata
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Database Engine-Level Immutability Enforcement
CREATE RULE deny_update_fee_transactions AS
ON UPDATE TO fee_transactions
DO INSTEAD NOTHING;

CREATE RULE deny_delete_fee_transactions AS
ON DELETE TO fee_transactions
DO INSTEAD NOTHING;

-- Extracted Entities Table for Named Entity Recognition Output
Integration
CREATE TABLE extracted_entities (
    entity_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(tenant_id) ON DELETE
CASCADE,
    entity_type VARCHAR(100) NOT NULL,
    text TEXT NOT NULL,
    source_type VARCHAR(100) NOT NULL, -- e.g., 'student_note',
'scurriculum_doc'
    source_id UUID NOT NULL,
    start_index INT NOT NULL,
    end_index INT NOT NULL,
    confidence REAL NOT NULL CHECK (confidence >= 0.0 AND confidence
<= 1.0),
    attributes JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

Multi-Tenant Isolation and Row-Level Security

To handle scaling across thousands of isolated PostgreSQL tenant schemas without exhausting database connection limits, PgBouncer is deployed in transaction mode. To prevent cross-tenant data leakage within shared connection pools, the application context wraps every transaction in a context-setting command, restricting the executing session before any query run :

```

SET LOCAL app.current_tenant_id =
'550e8400-e29b-41d4-a716-446655440000';

```

Row-Level Security (RLS) is enabled directly on all tenant-facing transactional tables to reject queries where the executing context lacks appropriate cryptographic assertions :

```
ALTER TABLE fee_transactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE extracted_entities ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY fee_transactions_isolation_policy ON fee_transactions
    AS RESTRICTIVE
    USING (tenant_id = NULLIF(current_setting('app.current_tenant_id',
true), '')::UUID);
```

```
CREATE POLICY extracted_entities_isolation_policy ON
extracted_entities
    AS RESTRICTIVE
    USING (tenant_id = NULLIF(current_setting('app.current_tenant_id',
true), '')::UUID);
```

GIN Operator Class Storage and Performance Trade-offs

When indexing JSONB structures for pattern-matching or containment searches within PostgreSQL, the selection of the correct GIN operator class has major implications on index size and write throughput.

Evaluation Dimension	GIN Operator Class: jsonb_ops (Default)	GIN Operator Class: jsonb_path_ops
Supported Operators	@>, ?, ? , ?&, @?, @@	@>, @?, @@
Indexing Mechanism	Creates individual index entries for every key, path, and value inside the JSONB document.	Hashes the full path along with the value, creating a single hashed key in the index structure.
Index Footprint	Extremely large; index size can reach 60% to 80% of the physical table size under complex schemas.	Highly compact; index size is typically 20% to 30% of the table size (2x to 3x smaller than default).
Insertion Performance Impact	High write latency; updates and inserts must decompose the entire JSONB tree to modify index lists.	Moderate write latency; hashing paths results in fewer index write operations during record updates.
Optimal Use Cases	Volatile schemas where queries search for arbitrary keys or check key existence (e.g., using ?).	Stable schemas with nested structures where queries rely purely on containment matching (e.g., @>).

```
-- DDL for Containment
Optim[span_3](start_span)[span_3](end_span)[span_6](start_span)[span_6]
](end_span)[span_9](start_span)[span_9](end_span)ization (Fastest
Search on Extracted Metadata)
CREATE INDEX CONCURRENTLY idx_entities_attributes_path_ops
```

```
ON extracted_entities
USING GIN (attributes jsonb_path_ops);

-- DDL for Flexible Key Existence Inquiries
CREATE INDEX CONCURRENTLY idx_entities_attributes_default_ops
ON extracted_entities
USING GIN (attributes);
```

Analytical Data Extraction Hierarchy

The data pipeline processes operational transactions, moving them through a structured extraction hierarchy to transform raw records into regulatory-compliant datasets.

Extraction Level	Output and Operational Focus
Level 0: Raw Database	Foundational PostgreSQL tables and Valkey key-value stores.
Level 1: Business Entity	Aggregated domain objects (e.g., complete Student Profile with encrypted PII).
Level 2: Relationships	Foreign keys and edge traversal across modules (e.g., Student to Enrolled Subjects).
Level 3: Historical Timeline	Event-sourced projections displaying state at exact temporal coordinates.
Level 4: Behavior	Time-series analysis of attendance patterns, assignment completion, and payment cadences.
Level 5: Prediction	Machine learning inference models scoring default probabilities or dropout risks.
Level 6: Recommendations	Algorithmic suggestions for timetable optimization and substitute teacher allocation.
Level 7: Automation	Orchestrated workflows, such as automatic late fee application without human intervention.
Level 8: Intel	Macro-level executive KPIs comparing campus performance metrics.
Level 9: Policy Intelligence	Auditing compliance against the National Education Policy 2020 or Karnataka SEP 2025.
Level 10: National Intel	Generation of formatted XML payloads for state and national portals (e.g., UDISE+, NSP).

Service Mesh, mTLS and Zero-Trust Network Fabric

The internal network is treated as hostile, necessitating mutual Transport Layer Security (mTLS v1.3) for all inter-service communication within the Kubernetes cluster. Selecting the appropriate service mesh requires balancing resource constraints, operational complexity, and latency requirements.

Service Mesh Performance Evaluation

Below is a structured analysis of service mesh performance based on independent benchmarks under a sustained traffic load of 2,000 requests per second (RPS).

Performance Dimension	Linkerd (Rust Proxy)	Istio (Envoy Proxy)	Consul (Envoy Proxy)
Median (P50) Added Latency	6 ms	15 ms to 17 ms	Balanced under light loads; degrades at high RPS
Tail (P99) Added Latency	16.5 ms	18.5 ms (Sidecar Mode) / 6.8 ms (Ambient Mode)	55.5 ms (at 1,200 RPS)
Maximum Added Latency	42 ms	250 ms to 350 ms	125.5 ms
Memory Footprint per Pod	15 MB to 26 MB	50 MB to 150 MB+ depending on config	~30 MB
Proxy CPU Overhead	10 ms CPU time	88 ms CPU time	Substantial under high traffic load
Control Plane Memory	365 MB	597 MB to 1+ GB	Balanced; ties into HashiCorp ecosystem
Security and Authorization	Lightweight identity-based access control via Server/HTTPRoute CRDs	Granular policies matching JWT claims and external authentication sources	Unified via Consul Intentions and Vault PKI integrations

Linkerd is selected as the default service mesh due to its lightweight Rust-based proxy design, which minimizes resource consumption. In multi-hop microservice topologies, the total added network latency can be modeled using the formula:

$$L_{\text{total}} = \sum_{i=1}^H (L_{\text{service}_i} + 2 \cdot L_{\text{proxy}})$$

where H represents the number of service-to-service hops, L_{service_i} is the internal processing latency of the target microservice, and L_{proxy} is the latency added by the sidecar proxy. Linkerd's sub-millisecond data-plane latency minimizes the compounding effect of network hops compared to heavier alternatives.

Linkerd mTLS and Authorization Policy Manifests

To protect sensitive AI endpoints, communication with the NER-Service is restricted using standard Linkerd and Gateway API CRDs.

```
apiVersion: policy.linkerd.io/v1beta3
kind: Server
metadata:
  name: ner-service-server
  namespace: core
spec:
  podSelector:
    matchLabels:
      app: ner-service
  port: 8080
  proxyProtocol: HTTP/2
```



```

---
apiVersion: policy.linkerd.io/v1alpha1
kind: MeshTLSAuthentication
metadata:
  name: api-gateway-client-identity
  namespace: core
spec:
  identities:
    - "api-gateway.core.serviceaccount.identity.linkerd.cluster.local"
---
apiVersion: policy.linkerd.io/v1alpha1
kind: AuthorizationPolicy
metadata:
  name: restrict-ner-access-to-gateway
  namespace: core
spec:
  targetRef:
    group: policy.linkerd.io
    kind: Server
    name: ner-service-server
  requiredAuthenticationRefs:
    - name: api-gateway-client-identity
      kind: MeshTLSAuthentication
      group: policy.linkerd.io

```

CI/CD, Observability and Automated Compliance

The delivery lifecycle combines containerization and declarative GitOps patterns to enforce security checks at every stage of the release pipeline.

[Git Commit] --> --> --> -->

Code merged to the main branch triggers automated pipelines that run Static Application Security Testing (SAST) via Semgrep. Container images are built, scanned for vulnerabilities using Trivy, and deployed to Amazon Elastic Kubernetes Service (EKS). Database migrations run as pre-deployment hooks using Drizzle ORM; if a migration fails, the deployment aborts to prevent runtime state mismatches.

Distributed tracing is implemented using OpenTelemetry, aggregating spans, metrics, and logs into a centralized dashboard. To ensure compliance with DPDP regulations, logging interceptors run regex-based redaction algorithms to mask sensitive PII (such as Aadhaar numbers, phone numbers, and financial details) before the log streams are written to the datastore.

Data retention lifecycles are strictly managed based on legal mandates :

- Financial records (including payroll ledgers and fee receipts) are stored in immutable, append-only tables for a minimum of seven years to comply with statutory audits.
- Student behavioral telemetry, location tracking history, and application logs are rotated out of hot storage after twelve months to minimize the stored data footprint.

Named Entity Recognition Models and Machine Learning Infrastructure

Transformer-based models (such as BERT, RoBERTa, or DeBERTa) represent the state-of-the-art for Named Entity Recognition (NER), outperforming legacy conditional random field (CRF) models by capturing contextual semantic embeddings. However, straightforward containerized deployments of these models can lead to excessively large Docker images (often exceeding 8 GB) due to heavy deep-learning frameworks (such as PyTorch or TensorFlow) and GPU-dependent CUDA dependencies.

To optimize inference latency and control cloud infrastructure costs, the system exports fine-tuned transformer models to the ONNX (Open Neural Network Exchange) format. Using the quantized ONNX model inside a multi-stage Docker build reduces the final container size to approximately 500 MB and allows the service to run on optimized CPU architectures (utilizing Intel MKL or AMD Zen DNN libraries) instead of expensive GPUs.

Multi-Stage Production Dockerfile

The following Dockerfile separates the build dependencies from the runtime image to minimize security vulnerabilities and reduce the final container footprint.

```
# =====
# Stage 1: Build & Compilation Environment
# =====
FROM python:3.11-slim AS builder

WORKDIR /build

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Install the high-speed 'uv' package manager
RUN pip install --no-cache-dir uv

# Install and compile Python packages to the specified prefix
# directory
COPY requirements.txt.
RUN uv pip install --system --no-cache-dir --prefix=/install -r
requirements.txt

# =====
# Stage 2: Final Production Environment
# =====
FROM python:3.11-slim

WORKDIR /app
```

```

# Copy compiled python dependencies from builder stage
COPY --from=builder /install /usr/local

RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates \
    && rm -rf /var/lib/apt/lists/*

# Copy application code and the quantized ONNX model binary
COPY ./app ./app
COPY ./models/roberta-ner-quantized.onnx ./app/models/roberta-ner-quantized.onnx

# Enforce secure container user boundaries
RUN useradd -u 8888 appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8080

ENV PORT=8080
ENV PYTHONUNBUFFERED=1

CMD

```

FastAPI Asynchronous ONNX Inference Implementation

The following Python script implements the asynchronous FastAPI service, performing model warm-up at startup to prevent cold-start delays.

```

import os
import numpy as np
import onnxruntime as ort
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="EduCore-NER-Inference-Engine", version="1.0.0")

class NERInferenceRequest(BaseModel):
    text: str = Field(..., min_length=2, max_length=5000,
        example="Professor Smith teaches Math101.")

class EntitySpan(BaseModel):
    entity_class: str
    text: str
    confidence: float

class NERInferenceResponse(BaseModel):
    entities: list

```

```

# Global execution state holding the ONNX Runtime session
ort_session = None

@app.on_event("startup")
def initialize_model_session():
    global ort_session
    model_path = "./app/models/roberta-ner-quantized.onnx"

    # Configure optimized CPU execution options
    options = ort.SessionOptions()
    options.intra_op_num_threads = 2
    options.execution_mode = ort.ExecutionMode.ORT_SEQUENTIAL

    # Initialize the runtime session
    ort_session = ort.InferenceSession(model_path, options,
    providers=["CPUExecutionProvider"])

    # Run dummy warm-up inference to compile runtime paths
    dummy_input = np.zeros((1, 128), dtype=np.int64)
    ort_session.run(None, {
        "input_ids": dummy_input,
        "attention_mask": dummy_input
    })

@app.post("/api/v1/ner/extract", response_model=NERInferenceResponse)
async def process_ner_extraction(payload: NERInferenceRequest):
    if not ort_session:
        raise HTTPException(status_code=503, detail="ONNX Inference
session uninitialized.")

    try:
        # Mocking input mapping (the real implementation runs a
HuggingFace FastTokenizer)
        input_ids = np.ones((1, 128), dtype=np.int64)
        attention_mask = np.ones((1, 128), dtype=np.int64)

        # Execute model execution over CPU-bound execution paths
        raw_outputs = ort_session.run(None, {
            "input_ids": input_ids,
            "attention_mask": attention_mask
        })

        # Process output logits and map tokens back to span identities
        extracted_entities =
        return NERInferenceResponse(entities=extracted_entities)
    except Exception as err:
        raise HTTPException(status_code=500, detail=f"Inference

```

```
session failed: {str(err)}")
```

NER Model Architecture Trade-offs

Model selection balances accuracy metrics on held-out test sets against runtime latencies and operational resource costs. Extraction accuracy is tracked using standard metrics:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$
$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad F_1 = 2$$
$$\cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Model Architecture	F_1 Score Accuracy	In-Memory Model Size	Latency per 500 Tokens	Serving Infra Requirement
RoBERTa-Base (Quantized ONNX)	~91.2%	~120 MB	~14 ms	Minimal (optimized CPU instance)
BERT-Base-Unca sed (Standard)	~89.5%	~440 MB	~45 ms	Standard CPU / Minimal GPU
DistilBERT-Base	~86.8%	~260 MB	~8 ms	Ultra-lightweight CPU
GPT-4 (Zero-Shot API)	~93.4%	N/A (Cloud Hosted)	~1200+ ms	Dependent on external API availability

Feature Store Integration via Feast

To eliminate training-serving skew, the system implements Feast to manage, version, and serve machine learning features consistently. Feast decouples feature engineering pipelines, calculating historical features from the relational database (PostgreSQL) for training, and storing only the latest feature values in Redis to enable low-latency online lookups at inference time.

```
project: educore_intelligence
```

```
registry:
```

```
  registry_type: sql
```

```
  path:
```

```
postgresql://feast_registry_user:SecurePass881@postgres-ha.core.svc.cl
```

```
uster.local:5432/feast_registry_db
```

```
[span_103] (start_span) [span_103] (end_span) [span_104] (start_span) [span_104] (end_span)
```

```
provider: local
```

```
online_store:
```

```
  type: redis
```

```
  connection_string:
```

```
redis://redis-online-ha.core.svc.cluster.local:6379/0
```

```
[span_105] (start_span) [span_105] (end_span) [span_106] (start_span) [span_106] (end_span)
```

```
  ssl: true
```

```
materialization:
```

```
  online_write_batch_size: 10000 -- Writes rows in chunks to prevent  
memory exhaustion [span_107] (start_span) [span_107] (end_span)
```

```
pull_latest_features: true -- Restricts reads to retrieve only the
latest values [span_108] (start_span) [span_108] (end_span)
```

Event Sourcing, State Machines and Sagas

The architecture relies on Event Sourcing for critical state changes. Rather than updating a row to reflect a new balance, the system appends immutable event records (such as `PaymentInitiated`, `PaymentCaptured`, or `ConcessionApplied`) to the ledger. To mitigate the risk of projection lag—where read models temporarily fall behind write events—the system uses synchronous local reads for critical transactional boundaries, accepting eventual consistency only for macro-analytical dashboards.

Payment State Machine

The payment state machine enforces strict, one-way transitions directly at the application boundary to prevent structural bypass :

`\text{PENDING}` $\rightarrow$ `\text{INITIATED}` $\rightarrow$ `\text{PROCESSING}` $\rightarrow$ `\text{CAPTURED}` $\mid$ `\text{FAILED}` $\mid$ `\text{REFUNDED}`

Any attempt to bypass these states (such as jumping from `PENDING` to `CAPTURED` without an intermediary `INITIATED` record) triggers an immediate transaction rejection and generates an anomaly alert in the security event log.

Distributed Saga Transactions Orchestrated by Temporal.io

In a microservices architecture, executing distributed workflows across distinct database instances cannot rely on traditional database-level ACID transactions. For multi-step workflows (such as student admissions, which involve student profile creation, fee capture, and resource allocation), the system implements the Saga Pattern orchestrated by Temporal.io. If any step in the sequence fails, the orchestrator executes predefined compensating transactions in reverse order to return the system to a consistent state.

```
import asyncio
from datetime import timedelta
from temporalio import workflow
from dataclasses import dataclass
```

```
@dataclass
class StudentAdmissionData:
    tenant_id: str
    student_id: str
    amount_paise: int
    course_id: str
```

```
@workflow.defn
class StudentAdmissionWorkflow:
    @workflow.run
    async def run(self, data: StudentAdmissionData) -> dict:
```

```

compensating_actions =

# Configure exponential backoff retry policy for activities
standard_retry = {
    "initial_interval": timedelta(seconds=1),
    "backoff_coefficient": 2.0,
    "maximum_interval": timedelta(seconds=30),
    "maximum_attempts": 3
}

try:
    # Step 1: Create Student Profile
    await workflow.execute_activity(
        "create_student_profile_activity",
        data,
        start_to_close_timeout=timedelta(seconds=10),
        retry_policy=standard_retry
    )

compensating_actions.append("delete_student_profile_compensation")

    # Step 2: Book Fee Transaction
    await workflow.execute_activity(
        "book_fee_transaction_activity",
        data,
        start_to_close_timeout=timedelta(seconds=10),
        retry_policy=standard_retry
    )

compensating_actions.append("reverse_fee_transaction_compensation")

    # Step 3: Allocate Hostel & Library Resources
    await workflow.execute_activity(
        "allocate_hostel_library_resources",
        data,
        start_to_close_timeout=timedelta(seconds=15),
        retry_policy=standard_retry
    )

compensating_actions.append("deallocate_hostel_library_compensation")

    return {"status": "SUCCESS", "message": "Student
successfully admitted."}

except Exception as ex:
    # Run compensating actions in reverse order to roll back
the state
    workflow.logger.error(f"Saga execution failed: {str(ex)}").

```

```
Replaying compensations.")
    for compensation in reversed(compensating_actions):
        try:
            await workflow.execute_activity(
                compensation,
                data,
                start_to_close_timeout=timedelta(seconds=15)
            )
        except Exception as rollback_err:
            workflow.logger.critical(
                f"CRITICAL: Compensation {compensation}
failed: {str(rollback_err)}"
            )
            return {"status": "FAILED", "error": str(ex)}
```

Agentic Orchestration and Defensive Governance

The platform implements a specialized, multi-agent orchestration architecture to manage business automation without introducing security vulnerabilities. To mitigate the risks of excessive agency (OWASP LLM06) and malicious configuration, agents operate under strict constraints defined by the OWASP Agentic Security Framework.

[Agent Execution Pool] --(Prohibited)--> --(Enforced)--> [Hand-Coded Logic]

To prevent hallucinatory logic from corrupting core operations, the "Absolute Laws of Vibe Coding" strictly prohibit AI agents from generating code related to fee calculations, payroll deductions, or tax arithmetic. All such logic is hand-coded as deterministic functions and verified with a minimum of 95% test coverage in the CI pipeline.

Multi-Agent Operational Governance Matrix

The execution limits and access permissions for each specialized agent are tightly scoped within bounded contexts.

AI Agent Designation	Primary Operational Responsibilities	Security Constraints & Risk Mitigation Policies
Product Research Agent	Analyzes market trends and competitor feature sets.	Denied access to internal production databases.
Compliance Agent	Evaluates DPDP, NIST, and state policy adherence.	Read-only access to regulatory knowledge graph.
Domain Architect	Designs entity-relationship models for new features.	Prohibited from executing database migrations unilaterally.
Security Architect	Reviews generated code against OWASP guidelines.	Mandatory human-in-the-loop review for all findings.
Frontend Architect	Generates React/Next.js UI components.	Output restricted to UI layer; cannot modify backend logic.

AI Agent Designation	Primary Operational Responsibilities	Security Constraints & Risk Mitigation Policies
Backend Architect	Constructs Fastify/Express API routes and schemas.	Prohibited from generating financial/payroll calculations.
Testing Agent	Generates unit and integration test coverage.	Limited token budget to prevent unbounded loop runs.
Deployment Agent	Drafts infrastructure-as-code manifests (Terraform).	Output requires cryptographic signatures from a human admin.

Antigravity IDE Workflow and Safety Gates

Developers use the Antigravity IDE workflow, which integrates these autonomous agents directly into the engineering lifecycle. To enforce architectural standards, the IDE's execution context is continuously fed a CONTEXT.md file containing the absolute laws of the system architecture. If an agent attempts to generate a database migration script or a cryptographic function, the IDE intercepts the output and flags it for manual developer intervention. To prevent Denial of Wallet (DoW) attacks caused by unbounded loops or malicious prompts, the orchestration layer implements strict token budget management. Once a defined token threshold is crossed, execution stops, requiring a human operator to investigate the anomaly and authorize additional compute resources. Furthermore, the Agent Communication Protocol cryptographically signs all inter-agent messages. Payloads are structurally validated to verify that no executable commands are embedded within semantic text transfers, protecting the system against prompt injection or cross-agent context poisoning.

Implementation Roadmap and Transition Phasing

Migrating the system to this cloud-native, microservices-driven architecture is structured in five distinct, dependency-mapped phases.

Phase / Timeline	Core Objectives & Deliverables	Estimated Effort (p-weeks)	Risk Mitigation and Contingency Strategies
Phase 1: Foundations (Weeks 1–6)	Deploy baseline Kubernetes infrastructure (EKS Fargate) in isolated VPC subnets. Set up PgBouncer and HashiCorp Vault. Create core relational tables.	4 p-weeks DevOps; 2 p-weeks per service.	Risk: Improper service decomposition. Mitigation: Enforce domain-driven design workshops to map bounded contexts.
Phase 2: Microservices & Mesh (Weeks 7–14)	Migrate operational services (Admissions, Finance, Attendance). Deploy the Linkerd service mesh and configure mTLS. Set up GitOps pipelines.	3 p-weeks DevOps; 2 p-weeks per service.	Risk: Environment drift and deployment failures. Mitigation: Enforce declarative Terraform/Helm files with automated validation.

Phase / Timeline	Core Objectives & Deliverables	Estimated Effort (p-weeks)	Risk Mitigation and Contingency Strategies
Phase 3: AI & Feast Ingestion (Weeks 15–24)	Fine-tune the RoBERTa NER model. Deploy the ONNX inference service. Configure the Feast Feature Store Operator and connect it to Redis.	4 p-weeks ML Eng; 3 p-weeks DB Eng.	Risk: Training data scarcity or low NER accuracy. Mitigation: Combine active learning pipelines with deterministic gazetteers.
Phase 4: Scaling & Hardening (Weeks 25–30)	Optimize database indexes and configure read-replicas. Enforce Linkerd mTLS security policies. Conduct compliance audits.	4 p-weeks DevOps; 2 p-weeks Security.	Risk: Scale bottlenecks during bulk traffic writes. Mitigation: Conduct end-to-end stress tests using Locust to scale pod sizing.
Phase 5: Continuous Evolution (Ongoing)	Monitor model drift and refine predictions. Automate UDISE+ and NSP reporting pipelines. Issue W3C verifiable credentials to DigiLocker.	Continuous; ~1 p-week/month.	Risk: Regulatory API modifications. Mitigation: Decouple compliance pipelines with adaptive mapping abstraction layers.

Assumptions, Trade-offs and Strategic ROI Analysis

The architecture is designed to scale horizontally to support medium-to-large educational institutions, assuming a baseline load of ~10,000 daily active users and requiring sub-second user interface latencies.

Architectural Assumptions and Target Scales

10K DAU ---> 100K+ DAU

- PostgreSQL (Single Node) Replicated	- PostgreSQL Sharded & Replicated
- Single Region EKS	- Multi-Region Active-Active EKS
- Standard Valkey Cache Cluster	- Distributed Redis Sentinel Cluster

- **Moderate Scale (Baseline):** Assumes ~10,000 daily active users with sub-second latency targets. If scale or budgets are tight, the deployment can run on simplified, single-node configurations to avoid the operational overhead of managing multi-tenant Kubernetes clusters.
- **Large Scale (Enterprise):** At scales exceeding 100,000 daily active users, the architecture dynamically scales horizontally by introducing multi-region EKS clusters, content delivery networks (CDNs), and database sharding or replication.

System Architectural Trade-Off and Decision Framework

Each core component is selected to optimize operational efficiency and maximize return on investment (ROI).

System Component	Evaluated Alternatives	Selected Architecture	ROI Justification & Trade-off Analysis
Service Mesh	Linkerd vs. Istio Sidecar vs. Istio Ambient	Linkerd (Rust Proxy)	Minimizes data-plane memory overhead (15-25 MB per pod) and added latency, reducing cloud compute costs.
Primary Database	PostgreSQL vs. MariaDB vs. Aurora Serverless	PostgreSQL 17	Mature support for row-level security (RLS), GIN document indexing, and schema-per-tenant isolation.
Inference Runtime	PyTorch vs. TensorFlow Serving vs. ONNX Runtime	ONNX Runtime (CPU Quantized)	Reduces container footprint from 8 GB to ~500 MB, allowing models to run on cost-effective CPU instances.
Distributed Transactions	Two-Phase Commit (2PC) vs. Saga Orchestration	Saga (Temporal.io Orchestrated)	Ensures data consistency across microservices without the blocking resource locks of 2PC.
Feature Store Storage	SQLite vs. Postgres Online vs. Redis Online	Redis (Online) + Postgres (Registry)	Guarantees sub-10ms feature retrieval at inference time, eliminating training-serving skew.

Conclusions and Actionable Recommendations

The proposed architectural modernization of the EduCore ERP platform establishes a secure, highly scalable, and AI-native ecosystem tailored for the Indian educational sector. By moving away from tightly coupled relational databases to a hybrid storage model (PostgreSQL for transactions, Neo4j for relationship mapping, and Feast for feature serving), the design resolves the performance limitations of traditional ERP systems. Enforcing bounded contexts via asynchronous EventStoreDB communication patterns eliminates cascading failures between domains, protecting financial operations from academic data anomalies.

From a security and compliance perspective, implementing a zero-trust network topology via Linkerd's mTLS v1.3 combined with database-level Row-Level Security guarantees isolation in multi-tenant environments and satisfies the legal mandates of the DPDP Act 2025. Furthermore,

optimizing machine learning deployments using quantized ONNX runtimes reduces serving costs, allowing the platform to run efficiently on standard CPU instances instead of expensive GPU clusters. Transitioning through the phased roadmap ensures that the engineering team can systematically mitigate risks while building a resilient, future-proof platform.

Works cited

1. How to Use Linkerd for mTLS - OneUptime, <https://oneuptime.com/blog/post/2026-01-26-linkerd-mtls/view>
2. PostgreSQL GIN Indexes: JSONB, Arrays & Full-Text Search - DEV Community, https://dev.to/philip_mcclarence_2ef9475/postgresql-gin-indexes-jsonb-arrays-full-text-search-29i2
3. PostgreSQL JSONB Operator Classes of GIN Indexes and Their Usage | by Josef Machytka, <https://medium.com/@josef.machytka/postgresql-jsonb-operator-classes-of-gin-indexes-and-their-usage-0bf399073a4c>
4. PostgreSQL JSONB Indexing: GIN, Expression & Partial Index Strategies - DEV Community, https://dev.to/philip_mcclarence_2ef9475/postgresql-jsonb-indexing-gin-expression-partial-index-strategies-i11
5. Latency Benchmarks: Istio, Linkerd, Consul | Hokstad Consulting, <https://hokstadconsulting.com/blog/latency-benchmarks-istio-linkerd-consul>
6. Istio vs. Linkerd: Which Service Mesh to Choose? - Plural, <https://www.plural.sh/blog/istio-vs-linkerd-guide/>
7. How to Compare Istio vs Linkerd for Your Use Case - OneUptime, <https://oneuptime.com/blog/post/2026-02-24-how-to-compare-istio-vs-linkerd-for-your-use-case/view>
8. Benchmarking Linkerd and Istio: 2021 redux | CNCF, <https://www.cncf.io/blog/2021/12/17/benchmarking-linkerd-and-istio-2021-redux/>
9. Istio vs Cilium vs Linkerd 2026: Service Mesh Comparison - Luca Berton, <https://lucaberton.com/blog/service-mesh-istio-ambient-cilium/>
10. Authorization Policy | Linkerd, <https://linkerd.io/2-edge/reference/authorization-policy/>
11. Configuring Per-Route Authorization Policy | Linkerd, <https://linkerd.io/docs/tasks/configuring-per-route-policy/>
12. Reducing the Size of Docker Images Serving Large Language Models, <https://towardsdatascience.com/reducing-the-size-of-docker-images-serving-llm-models-b70ee66e5a76/>
13. Fine-Tuning a Pre-Trained RoBERTa Model to Improve Performance on Domain Specific Sentiment Analysis Task | by Parker Moesta | Medium, https://medium.com/@parkermo_86729/fine-tuning-a-pre-trained-roberta-model-to-improve-performance-on-domain-specific-sentiment-e6fcd89a1546
14. Slimmer FastAPI Docker Images with Multi-Stage Builds - David Muraya, <https://davidmuraya.com/blog/slimmer-fastapi-docker-images-multistage-builds/>
15. Building AI-Powered APIs with FastAPI and OpenAI Agents SDK: Deployment on Hugging Face | by Daniel Hashmi | Dev Genius, <https://blog.devgenius.io/building-ai-powered-apis-with-fastapi-and-openai-agents-sdk-deployment-on-hugging-face-2ce34d3eb766>
16. FastAPI Docker Deployment: Preparing ONNX AI Models for AWS Lambda - PyImageSearch, <https://pyimagesearch.com/2025/11/17/fastapi-docker-deployment-preparing-onnx-ai-models-for-aws-lambda/>
17. jaketae/fastapi-bert - GitHub, <https://github.com/jaketae/fastapi-bert>
18. Feast Operator Helm Chart - Artifact Hub, <https://artifacthub.io/packages/helm/feast-operator/feast-operator>
19. Deploy Feast Feature Store on Kubernetes: Production MLOps Guide (2026) | NomadX, <https://kubernetes.ae/deploy-feast-feature-store-kubernetes/>
20. How to Run a Feature Store with Feast on Kubernetes for ML Feature Serving - OneUptime,

<https://oneuptime.com/blog/post/2026-02-09-feast-feature-store-kubernetes/view> 21. How to Setup Feast Feature Store on Kubernetes - DevOps Cube,
<https://devopscube.com/setup-feature-store-feast-on-kubernetes/> 22. Introduction to Feast: Building fast feature stores with Redis,
<https://redis.io/blog/building-feature-stores-with-redis-introduction-to-feast-with-redis/> 23. Managing the complexity of distributed transactions with Temporal.io - Medium,
<https://medium.com/twodigits/managing-the-complexity-of-distributed-transactions-with-temporal-io-3ebfaef6ce31> 24. Build a trip booking application in Python - Learn Temporal,
<https://learn.temporal.io/tutorials/python/trip-booking-app/> 25. Saga Compensating Transactions - Temporal,
<https://temporal.io/blog/compensating-actions-part-of-a-complete-breakfast-with-sagas> 26. Postgres vs Neo4j : Key Differences Explained - PuppyGraph,
<https://www.puppygraph.com/learn/postgres-vs-neo4j> 27. Knowledge graph generation - Neo4j,
<https://neo4j.com/blog/developer/knowledge-graph-generation/> 28. Automatic mTLS - Linkerd,
<https://linkerd.io/2-edge/features/automatic-mtls/>